

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Решение СЛАУ методом Гаусса

Студентка гр. 4342

Волкова В.Н.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Задание

Необходимо реализовать программу для решения системы линейных алгебраических уравнений методом Гаусса с частичным выбором главного элемента. Программа должна уметь формировать матрицы нескольких типов, находить решение системы, вычислять относительную невязку и число обусловленности, а также сравнивать полученный результат с библиотечной функцией `scipy.linalg.solve`.

Теоретические положения

Рассматривается система линейных уравнений $A \cdot x = b$, где A — квадратная матрица коэффициентов, x — вектор неизвестных, b — вектор свободных членов. Метод Гаусса сводит исходную систему к эквивалентной верхнетреугольной системе, после чего неизвестные находятся обратной подстановкой.

Алгоритм состоит из двух основных этапов. На прямом ходе последовательно выбирается ведущий элемент в текущем столбце, при необходимости строки переставляются, ведущая строка нормируется и с ее помощью зануляются элементы ниже диагонали. На обратном ходе неизвестные вычисляются снизу вверх, начиная с последнего уравнения.

В работе используется частичный выбор главного элемента по столбцу. На k -м шаге среди строк с номерами от k до n выбирается элемент с максимальным модулем в k -м столбце. Такая перестановка уменьшает влияние ошибок округления и делает вычисления устойчивее по сравнению с наивным вариантом метода.

Для оценки качества решения вычисляется относительная невязка: $\delta = \|A \cdot \hat{x} - b\|_2 / \|b\|_2$. Чем меньше это значение, тем лучше найденный вектор $\hat{x}$ удовлетворяет исходной системе.

Дополнительно анализируется число обусловленности матрицы $\text{cond}(A) = \|A\|_2 \cdot \|A^{-1}\|_2$. Оно показывает, насколько чувствительно решение к малым изменениям исходных данных. Если $\text{cond}(A)$ близко к 1, задача хорошо

обусловлена. Если $\text{cond}(A)$ очень велико, даже небольшая погрешность в коэффициентах может сильно изменить ответ.

Описание программы

Программа написана на языке Python с использованием библиотек NumPy и SciPy. Реализация метода Гаусса вынесена в отдельную функцию `gauss_partial_pivot(a, b)`, в которой выполняются перестановка строк, нормировка ведущей строки и обнуление элементов под диагональю.

Для генерации входных данных предусмотрены три типа матриц: случайная матрица, матрица Гильберта и диагонально-доминантная матрица. Для матрицы Гильберта дополнительно задается точное решение $x = (1, 1, \dots, 1)$, чтобы можно было оценить не только невязку, но и фактическую ошибку решения.

После вычисления решения методом Гаусса результат сравнивается с функцией `scipy.linalg.solve`. В отчете используются следующие показатели: относительная невязка, число обусловленности и относительное отличие полученного решения от библиотечного.

Программа поддерживает два режима запуска: `solve` — решение одной системы, и `research` — проведение серии вычислительных экспериментов для выбранного типа матрицы.

Выполнение работы

В ходе выполнения лабораторной работы был реализован метод Гаусса с частичным выбором главного элемента. Прямой ход построен так, чтобы на каждом шаге выбирался наибольший по модулю элемент в текущем столбце, после чего выполнялись перестановка строк и исключение неизвестной из нижележащих уравнений.

Для проверки корректности сначала была решена наглядная система размером 3×3 , для которой известен простой ответ. Затем были проведены вычислительные эксперименты для случайных, диагонально-доминантных и гильбертовых матриц.

При анализе результатов учитывалось, что малая невязка еще не всегда означает малую ошибку самого решения. Особенно заметно это на плохо обусловленных матрицах Гильберта: система формально удовлетворяется очень хорошо, но найденный вектор может заметно отличаться от точного.

Результаты тестирования

Тестовая система: $2x_1 + x_2 - x_3 = 8$; $-3x_1 - x_2 + 2x_3 = -11$; $-2x_1 + x_2 + 2x_3 = -3$.

Таблица 1 – Проверка реализации на системе 3×3

Переменная	Метод Гаусса	SciPy	Разность
x_1	2.000000	2.000000	-4.44e-16
x_2	3.000000	3.000000	4.44e-16
x_3	-1.000000	-1.000000	-1.11e-15

Для данной системы получено точное решение $x = (2, 3, -1)$. Относительная невязка составила 0.00e+00, число обусловленности матрицы — 44.67. Метод Гаусса и библиотечное решение полностью совпали.

Таблица 2 – Результаты для случайных и диагонально-доминантных матриц

Тип матрицы	n	Относительная невязка	cond(A)	Отличие от SciPy
Случайная	10	3.87e-16	6.97e+01	5.45e-16
Случайная	100	8.77e-15	1.48e+03	1.75e-14
Случайная	500	1.81e-14	4.54e+04	4.67e-14
Диагонально-доминантная	10	7.78e-17	3.99e+00	4.79e-17
Диагонально-доминантная	100	4.37e-16	2.38e+00	7.09e-16
Диагонально-доминантная	500	8.31e-16	2.20e+00	1.40e-15

Для случайных матриц число обусловленности оказалось умеренным и возрастало вместе с размером матрицы в проведенном эксперименте. Невязка и отличие от SciPy оставались очень малыми. Для диагонально-доминантных матриц cond(A) оказался близок к 1–4, что соответствует хорошо обусловленной задаче и очень устойчивому решению.

Таблица 3 – Результаты для матриц Гильберта

n	Относительная невязка	cond(A)	Отличие от SciPy	Максимальная ошибка	Комментарий
---	-----------------------	---------	------------------	---------------------	-------------

5	7.91e-17	4.77e+05	6.64e-13	9.33e-13	ошибка практически незаметна
8	8.03e-17	1.53e+10	2.08e-07	2.73e-07	ошибка практически незаметна
10	1.13e-16	1.60e+13	1.08e-06	2.25e-05	ошибка практически незаметна
12	1.20e-16	1.64e+16	3.81e-02	1.93e-01	решение уже заметно искажено

Для матриц Гильберта уже при сравнительно малых размерах число обусловленности становится огромным. Интересно, что относительная невязка остается очень маленькой, однако максимальная ошибка решения быстро растет. Это подтверждает, что на плохо обусловленных задачах одной только невязки недостаточно для оценки качества найденного вектора.

Анализ результатов

Проведенные вычисления показывают, что устойчивость решения определяется не только размером матрицы, но в первую очередь ее структурой. Диагонально-доминантные матрицы решаются наиболее надежно: число обусловленности мало, а результаты практически совпадают с библиотечными.

Случайные матрицы в выбранных экспериментах также решались корректно, но их $\text{cond}(A)$ оказался заметно выше. Из-за этого невязка и расхождение с эталонным решением несколько увеличиваются, хотя остаются в пределах машинной точности.

Наиболее показательными оказались матрицы Гильберта. Для них даже при $n = 10-12$ наблюдается очень плохая обусловленность. Из-за этого маленькие ошибки округления во время вычислений усиливаются, и фактическая ошибка решения становится существенной. Следовательно, при анализе таких систем необходимо смотреть не только на невязку, но и на обусловленность матрицы.

Вывод

В ходе лабораторной работы был реализован метод Гаусса с частичным выбором главного элемента для решения СЛАУ. Корректность реализации подтверждена сравнением с функцией `scipy.linalg.solve`.

Проведенное исследование показало, что метод дает точные и устойчивые результаты для хорошо обусловленных матриц, в частности для диагонально-доминантных систем. Для случайных матриц точность также остается высокой, но зависит от конкретной структуры коэффициентов.

Для матриц Гильберта было показано, что при очень большом числе обусловленности даже малая невязка не гарантирует малую ошибку решения. Таким образом, при решении СЛАУ важно контролировать не только сам ответ, но и свойства матрицы системы.

Приложение А

Исходный код программы

Файл: gauss_method.py

```
import argparse
import numpy as np
import scipy.linalg

def parse_args():
    parser = argparse.ArgumentParser(
        description="Решение СЛАУ методом Гаусса с частичным выбором главного
элемента"
    )

    subparsers = parser.add_subparsers(dest="command", required=True)

    research_parser = subparsers.add_parser("research", help="Провести серию
экспериментов")
    research_parser.add_argument("--n", type=int, required=True, help="Размер
матрицы")
    research_parser.add_argument(
        "--type",
        choices=["random", "hilbert", "diagonal-dominant"],
        default="random",
        help="Тип матрицы"
    )
    research_parser.add_argument("--interval", type=float, default=100.0,
help="Верхняя граница генерации")
    research_parser.add_argument("--seed", type=int, default=1, help="Начальное
значение генератора")

    solve_parser = subparsers.add_parser("solve", help="Решить одну систему")
    solve_parser.add_argument("--n", type=int, required=True, help="Размер
матрицы")
    solve_parser.add_argument(
        "--type",
        choices=["random", "hilbert", "diagonal-dominant"],
        default="random",
        help="Тип матрицы"
    )
    solve_parser.add_argument("--interval", type=float, default=100.0,
help="Верхняя граница генерации")
    solve_parser.add_argument("--seed", type=int, default=1, help="Начальное
значение генератора")

    return parser.parse_args()

def validate_args(args):
    if args.n < 2:
        raise ValueError("Размер матрицы должен быть не меньше 2")
    if args.interval <= 0:
        raise ValueError("Интервал генерации должен быть положительным")

def generate_random_system(n, interval=100.0, seed=1):
    rng = np.random.default_rng(seed)
    a = rng.uniform(0, interval, size=(n, n))
    b = rng.uniform(0, interval, size=n)
    return a, b
```

```

def generate_hilbert_system(n):
    idx = np.arange(1, n + 1)
    a = 1.0 / (idx[:, None] + idx[None, :] - 1)
    x_exact = np.ones(n)
    b = a @ x_exact
    return a, b, x_exact

def generate_diagonal_dominant_system(n, interval=100.0, seed=1):
    rng = np.random.default_rng(seed)
    a = rng.uniform(0, interval, size=(n, n))
    for i in range(n):
        row_sum = np.sum(np.abs(a[i])) - abs(a[i, i])
        a[i, i] = row_sum + rng.uniform(1.0, interval)
    b = rng.uniform(0, interval, size=n)
    return a, b

def gauss_partial_pivot(a, b):
    a = a.astype(float).copy()
    b = b.astype(float).copy()
    n = len(b)

    for k in range(n):
        pivot_row = k + np.argmax(np.abs(a[k:, k]))
        if abs(a[pivot_row, k]) < 1e-15:
            raise ValueError("Матрица вырождена или близка к вырожденной")

        if pivot_row != k:
            a[[k, pivot_row]] = a[[pivot_row, k]]
            b[[k, pivot_row]] = b[[pivot_row, k]]

        pivot = a[k, k]
        a[k, k:] /= pivot
        b[k] /= pivot

        if k + 1 < n:
            factors = a[k + 1:, k].copy()
            a[k + 1:, k:] -= factors[:, None] * a[k, k:]
            b[k + 1:] -= factors * b[k]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = b[i] - a[i, i + 1:] @ x[i + 1:]
    return x

def relative_residual(a, x, b):
    numerator = np.linalg.norm(a @ x - b, ord=2)
    denominator = np.linalg.norm(b, ord=2)
    return numerator / denominator if denominator != 0 else numerator

def relative_difference(x1, x2):
    return np.linalg.norm(x1 - x2, ord=2) / np.linalg.norm(x2, ord=2)

def analyze_system(a, b):
    x_gauss = gauss_partial_pivot(a, b)
    x_scipy = scipy.linalg.solve(a, b)
    return {
        "gauss_solution": x_gauss,

```



```

        "scipy_solution": x_scipy,
        "residual": relative_residual(a, x_gauss, b),
        "condition_number": np.linalg.cond(a),
        "difference": relative_difference(x_gauss, x_scipy),
    }

def make_system(matrix_type, n, interval, seed):
    if matrix_type == "random":
        return generate_random_system(n, interval, seed), None
    if matrix_type == "diagonal-dominant":
        return generate_diagonal_dominant_system(n, interval, seed), None
    if matrix_type == "hilbert":
        a, b, x_exact = generate_hilbert_system(n)
        return (a, b), x_exact
    raise ValueError("Неизвестный тип матрицы")

def run_solve(args):
    (a, b), x_exact = make_system(args.type, args.n, args.interval, args.seed)
    result = analyze_system(a, b)

    print(f"Тип матрицы: {args.type}")
    print(f"Размер: {args.n}")
    print("-" * 70)
    print(f"{'i':<5}{'Метод Гаусса':<24}{'SciPy':<24}")
    print("-" * 70)
    for i, (g, s) in enumerate(zip(result["gauss_solution"],
result["scipy_solution"]), start=1):
        print(f"{'i':<5}{'g':<24.10f}{'s':<24.10f}")
        print("-" * 70)
    print(f"Относительная невязка: {result['residual']:.6e}")
    print(f"Число обусловленности: {result['condition_number']:.6e}")
    print(f"Относительное отличие от SciPy: {result['difference']:.6e}")

    if x_exact is not None:
        max_error = np.max(np.abs(result["gauss_solution"] - x_exact))
        print(f"Максимальная ошибка относительно точного решения:
{max_error:.6e}")

def run_research(args):
    sizes = [args.n]
    if args.type == "hilbert":
        print(f"{'n':<8}{'Невязка':<18}{'cond(A)':<18}{'Отличие от
SciPy':<20}{'Макс. ошибка':<18}")
        print("-" * 82)
        for n in sizes:
            (a, b), x_exact = make_system(args.type, n, args.interval, args.seed)
            result = analyze_system(a, b)
            max_error = np.max(np.abs(result["gauss_solution"] - x_exact))
            print(
f"{'n':<8}{result['residual']:<18.6e}{result['condition_number']:<18.6e}"
f"{'result['difference']:<20.6e}{max_error:<18.6e}"
)
    else:
        print(f"{'n':<8}{'Невязка':<18}{'cond(A)':<18}{'Отличие от SciPy':<20}")
        print("-" * 64)
        for n in sizes:
            (a, b), _ = make_system(args.type, n, args.interval, args.seed)
            result = analyze_system(a, b)
            print(

```

```
f"{n:<8}{result['residual']:<18.6e}{result['condition_number']:<18.6e}"
    f"{result['difference']:<20.6e}"
    )
```

```
def main():
    args = parse_args()
    validate_args(args)
    if args.command == "solve":
        run_solve(args)
    else:
        run_research(args)
```

```
if __name__ == "__main__":
    main()
```